

Mini HTML Parser and DOM Builder

Topic: PLY (Python Lex-Yacc)

Wed, Jan 8, 2026

1 Introduction

This assignment involves the design and implementation of a **mini HTML parser** for a restricted subset of the HTML language. The parser reads an HTML file, validates its syntax and semantics, constructs a DOM-like tree representation, and serializes the resulting structure into JSON format.

The goal of this assignment is to provide hands-on experience with lexical analysis, context-free grammar parsing, abstract syntax tree construction, and semantic validation.

2 Objectives

The primary objectives of this assignment are:

- Implement lexical analysis for a structured language
- Parse input using a context-free grammar
- Construct a hierarchical DOM-like tree
- Perform semantic validation of attributes

3 Supported HTML Subset

3.1 Supported Tags and Attributes

Only the following HTML tags and attributes are supported:

Tag	Supported Attributes
div	id, class
p	id, align, class
a	id, href, target, class

3.2 Attribute Rules

- Attributes are optional
- Attributes appear only in opening tags
- Attribute values must be enclosed in double quotes
- Attribute order is not significant
- Duplicate attributes on the same tag are syntax errors
- Attributes not listed for a tag must raise a parse-time error

3.3 Text Content Rules

- Any character sequence between tags is treated as a TEXT node
- Text nodes may include whitespace
- HTML entities, comments, scripts, and styles are not supported

4 Input Specification

- Input is read from a file named `input.html`
- The file contains HTML conforming to the supported subset

4.1 Example Input

```
<div id="root" class="container">
  <div id="header" class="section">
    <p class="text">Welcome</p>
  </div>
  <div id="content" class="section">
    <p id="intro" align="center">
      Hello <a href="https://example.com" target="_blank">world</a>
    </p>
  </div>
</div>
```

5 Output Specification

- Output must be written to a file named `output.json`
- The output represents the parsed DOM tree in JSON format

6 Node Structure

Each node in the output tree follows the structure below:

```
{
  "type": "ELEMENT" | "TEXT",
  "tagName": "string (optional)",
  "attributes": { "key": "value" },
  "children": [ Node ]
}
```

6.1 Notes

- `tagName` is present only for ELEMENT nodes
- `attributes` is an empty object if no attributes are present
- TEXT nodes contain their content as a childless node

7 Example Output

```
{
  "type": "ELEMENT",
  "tagName": "div",
  "attributes": {
    "id": "root",
    "class": "container"
  },
  "children": [
    {
      "type": "ELEMENT",
      "tagName": "div",
      "attributes": {
        "id": "header",
        "class": "section"
      },
      "children": [
        {
          "type": "ELEMENT",
          "tagName": "p",
          "attributes": {
            "class": "text"
          },
          "children": [
            {
              "type": "TEXT",
              "children": [],
              "value": "Welcome"
            }
          ]
        }
      ]
    }
  ],
  {
    "type": "ELEMENT",
    "tagName": "div",
    "attributes": {
      "id": "content",
      "class": "section"
    },
    "children": [
      {
        "type": "ELEMENT",
        "tagName": "p",
        "attributes": {
          "id": "intro",
          "align": "center"
        },
        "children": [
          {
            "type": "TEXT",
            "children": [],
            "value": "Hello "
          },
          {
            "type": "ELEMENT",
            "tagName": "a",
```

```
        "attributes": {
            "href": "https://example.com",
            "target": "_blank"
        },
        "children": [
            {
                "type": "TEXT",
                "children": [],
                "value": "world"
            }
        ]
    }
]
```

8 Functional Requirements

8.1 Lexer

The lexer must tokenize the input into meaningful tokens, including:

- Tag delimiters (<, >, </)
- Identifiers (tag names and attribute names)
- String literals (attribute values)
- Text content

8.2 Parser

- Implement recursive descent or table-driven parsing
- Construct a hierarchical tree representing the document structure
- Ensure proper nesting and matching of start and end tags

8.3 Semantic Validation

The parser must detect and report the following errors:

- Invalid or unsupported tags
- Invalid attributes for a given tag
- Duplicate attributes within a tag
- Missing closing tags
- Mismatched start and end tags
- Unquoted attribute values
- Unexpected end of file

Where possible, error messages should include line and column numbers.

9 Constraints

- Existing HTML or XML parsing libraries must not be used
- The problem must not be solved using regular expressions alone
- Use PLY (Python Lex-Yacc) only
- Standard libraries for file I/O and JSON serialization are allowed

10 Submission Requirements

Students must submit a single compressed archive with the following contents:

- **Source Code**
- **README.md** containing:
 - Grammar used
 - Parsing strategy
 - Data structures
- **Sample input.html**
- **Generated output.json**

Submission Details

- **Deadline:** Tue, Jan 13, 2026
- **Submission Format:** <roll_no>_PLY.zip